

The Six Provenance Metadata Fields per Substrate Write: The Architectural Infrastructure for Path Retracement in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 4 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the six provenance metadata fields per substrate write — three unconditional, three conditional — as a standalone architectural commitment that constitutes the metadata infrastructure on which the four path-retraceability accountability questions are answered.

Abstract

The CKS pattern's path-retraceability commitment requires substrate content to carry sufficient provenance that decisions can be answered from substrate alone — what was decided, by whom, under what authority, with what rationale. A separate set of derivation notes formalizes the four accountability questions; this note formalizes the substrate-side metadata infrastructure that supports them. Six fields constitute the commitment: three unconditional (writer attribution, timestamp, antecedent reference) required for every substrate write produced by deployment activity, and three conditional (rule reference, rationale, relationship to contradicting content) required when their architectural conditions hold. The conditional structure is itself part of the commitment: implementations that flatten it — treating all six as universally required, or treating any of the conditions as deployment-configurable — fail the commitment in the architectural sense even when their nominal field coverage is broader. The note defines each field, articulates the conditional structure as architectural rather than operational, distinguishes the commitment from four adjacent provenance patterns, names the load-bearing connections to downstream commitments, enumerates nine failure modes, and provides an operational test.

1. Why the six fields need standalone treatment

The parent note A1.07 commits CKS to path retraceability through the accountability vocabulary — what was decided, by whom, under what authority, with what rationale — and identifies the provenance fields substrate content must carry for the commitment to hold. The integrating-frame note A2.35 enumerates the six fields at the integrating level. A2.36, A2.37, A2.38, and A2.39 specialize the four accountability questions as standalone commitments. This note formalizes their substrate-side complement: the metadata infrastructure that the four questions are architecturally answerable through.

The motivating cases are deployments that capture some provenance fields but not all, leaving accountability questions partially answerable in ways that depend on which fields are missing rather than on

architectural commitment. Writes that carry writer and timestamp but no antecedent reference fragment paths at the missing field; cell writes that carry writer but no rule reference cannot answer "under what authority" architecturally for the entire cell-mediated class; contradicting writes that lack relationship metadata accumulate inconsistencies as content rather than carrying conflicts as first-class objects per A1.03 and A2.16. Each pattern is recurrent; each fails the commitment in a way invisible without the standalone treatment to compare against.

The relationship to A2.16 is specialization, not redundancy: A2.16 covers four conflict-specific fields (writer, timestamp, rationale, relationship) for contradicting content; A2.40 covers six fields applicable across all substrate writes, of which the four conflict-specific fields are a subset. The six-field structure with its conditional sub-structure is consequential prior art because it forecloses architectures that capture provenance partially or that flatten the conditional structure in either direction.

2. The six provenance metadata fields, defined precisely

In the CKS pattern, every substrate write produced by deployment activity carries a provenance record drawn from six fields. Three are unconditional — required for every such write. Three are conditional — required when specific architectural conditions hold.

Unconditional fields.

(a) Writer attribution. Identifies who or what produced the substrate content. The architecture distinguishes three writer categories: a human writing directly per A2.05, an LLM mediator operating under an orchestration rule per Property B from A2.20, and a non-LLM-mediated cell operating under an orchestration rule per A2.09. For human writers, the attribution links to the authority structure per A1.01.

(b) Timestamp. Identifies when the content was committed to substrate. The architectural commitment is to comparable timestamps that support temporal reconstruction; format and precision (clock time, logical clocks, monotonic sequence numbers) are deployment choices.

(c) Antecedent reference. Identifies the substrate content the writer read as input that informed the write. For cell-produced writes, it points to the substrate content the cell read within its bounded scope per A2.09. For direct human writes, it points to the substrate content the human consulted. Antecedent reference is what makes paths *retraceable through substrate* per A2.41 — without it, a write's causal antecedents are not part of substrate state, and path traversal terminates at the write.

Conditional fields.

(d) Rule reference. Identifies the orchestration rule that authorized the write. Required for cell-produced writes per Property B from A2.20, where the rule is the authority context. Not required for direct human writes per A2.05, where the human's authority position serves the role the rule serves for cells.

(e) Rationale. Captures the writer's reasoning for the decision the write records. Required where the orchestration rule that authorized the write or the substrate schema specifies rationale capture per A2.39. Not required otherwise.

(f) Relationship to contradicting content. Identifies the substrate content this write contradicts, with characterization of the contradiction's nature and bidirectional symmetry per A2.16. Required when the write contradicts other substrate content; not required for non-conflicting writes.

The six fields together constitute the substrate's provenance commitment. Implementations capture the three unconditional fields for every substrate write and the three conditional fields when their architectural conditions hold. Failing any required field breaks the accountability questions per A2.36–A2.39 the field supports.

3. The conditional structure as architectural

The conditional structure is the most operationally distinctive content of the six-field commitment, and the most commonly misread. Implementations under design pressure tend to flatten it in one of two directions: requiring all six fields universally (forcing placeholder content for fields whose conditions do not hold), or relocating the conditions into deployment policy (making the conditions configurable rather than architectural). Both flatten the commitment. Stating precisely why each conditional field's *condition itself* is architectural — and why deployments cannot relocate the conditions without breaking the commitment — is the work of this section.

Rule reference (field d). The condition is "the write was produced by a cell." Cells operate under orchestration rules per Property B from A2.20; the rule is the authority context that authorizes the cell's write. Direct human writes per A2.05 do not have a rule as their authority context — the human's position in the authority structure per A1.01 serves that role. Including a rule reference for a direct human write would either reintroduce the no-justification-as-precondition friction A2.05 explicitly forecloses for direct overrides, or require a placeholder rule reference that would degrade the field for cell writes by example. Conversely, omitting rule reference for cell writes would defeat Property B from A2.20: the cell's rule-authorization would not be substrate-recorded, and the "under what authority" question would fail architecturally for the entire cell-mediated class. The condition is architectural because it tracks which authority context applies, and the authority context is architectural per A2.20 and A1.01.

Rationale (field e). The condition is "the orchestration rule or substrate schema specifies rationale capture." Per A2.39, the architecture supports both rationale-rich and rationale-light deployments, with the requirement set by the orchestration rule that authorizes the write or by the substrate schema for the cell category the write belongs to. Mandatory rationale on every write would either burden routine operations with capture overhead the rule and schema have not asked for, or normalize placeholder rationale (defeating the field's purpose). Optional rationale at the deployment's discretion would relocate the requirement out of the rule and schema, where A2.39 grounds it. The condition is architectural because the rule and schema are architectural artifacts per A1.07's accountability-plan vocabulary; the requirement to capture rationale flows from them, not from local policy.

Relationship to contradicting content (field f). The condition is "the write contradicts other substrate content." Per A1.03 and A2.16, conflicts are first-class addressable objects with provenance attached to the conflict; the relationship field is what makes contradicting writes carry the conflict relationship as substrate content rather than as accumulated inconsistency. Non-conflicting writes have no contradicting content to relate to; mandatory relationship metadata for all writes would either require placeholder relationships or impose architectural overhead on writes that have no relationship to record. Omitting relationship for contradicting writes would break A1.03 — contradictions would persist as content but without the relationship that makes them addressable as conflicts. The condition is architectural because conflict status is architectural per A1.03; whether a write is contradicting or non-contradicting is a property of substrate content and its relations, not of deployment configuration.

The three conditions cannot be relocated into deployment policy without breaking the commitment, because they track architectural categories — cell-versus-human writer, rule-or-schema-required-rationale, contradicting-versus-non-contradicting content — that the architecture itself draws.

4. What the six-field commitment does not claim

The standalone treatment is not maximalist. Four items name what the commitment does not require, so the architectural floor is not mistaken for an architectural ceiling.

It does not exhaust useful provenance metadata. Deployments may add fields for operational tracking, debugging, or observability. The six are the floor, not the ceiling.

It does not specify field formats or schema implementation. Each field can be encoded in any form — structured records, identifier strings, references to other substrate content, cryptographic signatures — provided the encoding supports the field's architectural content. Implementation as columns, key-value pairs, or references to a separate provenance store (provided that store is itself substrate per A2.41) is a deployment choice.

It does not specify retention duration. Provenance fields persist as long as the substrate content they describe persists per A2.08. Absolute retention is a deployment concern.

It does not specify access patterns. Reading provenance fields may require substrate queries, metadata reading, or traversal logic; the architectural commitment is to the fields being substrate content accessible per A2.25, not to specific access performance.

5. What the six-field commitment is NOT

Four adjacent provenance patterns are commonly conflated with the six-field commitment. Each is a real commitment in some other architecture; naming what the commitment is not is what prevents the misreading.

Not audit logs. Audit logs record events external to substrate — which user accessed which content, when, from where. The six fields are different in kind: properties of substrate content itself, embedded as substrate state, visible to humans inspecting the substrate directly per A2.25. Audit logs may complement the six fields but cannot substitute for them, because the substrate's own state is what answers the accountability questions per A2.41.

Not change tracking systems. Change tracking records how substrate state changed over time — version-to-version deltas, sequences of edits. The six fields are properties of each write that exists in substrate now. A change tracking system that records deltas without provenance on the resulting state fails the commitment because the substrate's current state lacks the provenance.

Not diff records. Diff-based systems record what was added, removed, or modified. The six fields are not diff content; they are properties of the resulting substrate content, regardless of how the resulting state was reached.

Not version history metadata. Version control systems track metadata about versions — commit messages, version numbers, branch information. The six fields are not version metadata; they are provenance for substrate content in its current state. Version-controlled substrate may carry version metadata in addition to the six fields, but version metadata alone does not satisfy the commitment.

6. Load-bearing for downstream commitments

The six-field commitment is load-bearing for several CKS commitments. It supports **the four accountability questions per A2.36–A2.39**: "what was decided" through substrate content together with antecedent reference (field c); "by whom" through writer attribution (field a); "under what authority" through rule reference (field d) for cell writes or writer-attribution-to-authority-structure linkage for human writes; "with what rationale" through rationale (field e) where required. It supports **the substrate-only-paths property per A2.41**: path traversal requires antecedent reference at every step. It supports **the conflict-as-first-class commitment per A1.03 and A2.16**: relationship to contradicting content (field f) is what makes a contradicting write carry the conflict as substrate content addressable as a conflict. It supports **the AI-as-substrate-mediator commitment per A1.04 and the LLM-write properties per A2.20–A2.24**: rule reference (field d) is what makes Property B's rule-authorization architectural rather than procedural; writer attribution (field a) carries the LLM-specific writer identification per Property E from A2.23. And it supports **the architectural-property qualifier per A2.06**: provenance is architectural rather than procedural because the six fields are substrate content, queryable from substrate alone per A2.41. Provenance held outside substrate would make the accountability questions procedural, depending on external systems for answers the architecture commits substrate to.

7. Failure modes that violate the six-field commitment

Nine failure modes name the most common ways implementations fail the commitment. The first three name field-coverage failures the conditional structure makes describable. The next four name failures arising from deployment pressures — storage optimization, integration with external audit infrastructure, late-binding provenance pipelines, and substrate maintenance over long-lived deployments. The last two name field-omission failures that recur often enough to deserve standalone treatment.

(a) Missing unconditional fields. Implementation fails to capture writer attribution, timestamp, or antecedent reference for some writes. Accountability questions fail for the writes with missing fields.

(b) Missing conditional fields when conditions hold. Implementation captures the unconditional fields but fails to capture rule reference for cell writes, rationale where the rule or schema requires it, or relationship for contradicting writes. The conditional commitments are unenforced when their architectural conditions hold.

(c) Universal application of conditional fields. Implementation requires all six fields for every write, generating placeholder content for conditional fields whose conditions do not hold. The substrate carries placeholder fields that degrade the architectural meaning of the fields when their conditions do hold.

(d) External-system field placement. Some fields are held in systems external to substrate — audit-log databases, observability platforms, change-tracking infrastructure — rather than in substrate itself. This pattern recurs in deployments that integrate with enterprise audit infrastructure: provenance is captured, but in the enterprise audit system rather than in the CKS substrate. Substrate-only paths per A2.41 fail; the architectural-property qualifier per A2.06 degrades to procedural-property because the answer depends on the external system's availability and current policy. The deployment's intentions may be sound — enterprise audit infrastructure is designed for exactly the role of provenance retention — but the architectural commitment is to provenance *as substrate state*, and external-system placement breaks it regardless of how complete the external system's records are.

(e) Post-hoc field generation. Fields are generated after the write commits — by an asynchronous pipeline that scans substrate writes and reconstructs provenance, or by an enrichment process that backfills fields on a schedule. This pattern recurs in deployments that optimize the write path for throughput. The atomic-with-content commit per A2.10 is broken; there is a window during which substrate content exists without its provenance, and during that window the substrate's source-of-truth status fails for any accountability question that depends on the missing fields. Even when the pipeline eventually populates the fields, the temporal window violates the commitment because the architectural property is co-presence with content, not eventual consistency with content.

(f) Field degradation over time. Automated processes — substrate compression, schema normalization, archival migration, index rebuilding — strip or degrade provenance fields over time. This pattern recurs in long-lived deployments where storage optimization runs periodically and treats provenance fields as candidates for compaction. The substrate's commitment to fields persisting per A2.08 is eroded; older content carries weaker provenance than newer content, and accountability questions become time-dependent in their answerability. The degradation is often invisible at the write path — writes capture full provenance — and visible only when historical accountability questions are asked of older content, by which time the original deployment configuration that drove the degradation may itself be unrecoverable.

(g) Antecedent-reference omission. Writes record writer attribution and timestamp but not antecedent reference. This pattern recurs in deployments that treat provenance as "who and when" — a default shaped by audit-log conventions — without recognizing that "what informed this" is an architecturally distinct field. Paths through substrate cannot be traced back from writes to what the writer read; the substrate-only-paths property per A2.41 fails for path traversal, and the "what was decided" accountability question per A2.36 fails for any decision whose antecedents the reader needs to consult. Antecedent-reference omission is the most consequential single field-omission because it breaks path traversal across writes that themselves carry full attribution.

(h) Rule-reference omission for cells. Cell-produced writes record writer attribution but not the rule that authorized the write. The "under what authority" accountability question per A2.38 fails for cell writes; Property B from A2.20 fails because the rule-authorization is not substrate-recorded.

(i) Relationship omission for conflicts. Contradicting writes exist in substrate but lack relationship metadata. Conflicts are accumulated inconsistencies rather than first-class objects per A1.03 and A2.16; the conflict-as-first-class commitment fails for the conflicts the relationship field would have made addressable.

A system that exhibits any of (a)–(i) does not implement the six-field commitment, and naming the failure precisely is what allows downstream remediation.

8. Operational test

A substrate satisfies the six-field provenance commitment if and only if all of the following are true at all times during the substrate's existence.

1. Every piece of substrate content produced by deployment activity carries non-empty writer attribution, timestamp, and antecedent reference (the unconditional fields).
2. Cell-produced writes (LLM-mediated or non-LLM-mediated) carry rule reference per Property B from A2.20.

3. Substrate content where the orchestration rule that authorized the write or the substrate schema specifies rationale capture carries non-placeholder rationale per A2.39.
4. Substrate content that contradicts other substrate content carries relationship metadata per A2.16.
5. Direct human writes carry the unconditional three fields plus rationale where required, with no rule reference (the human's authority position per A1.01 replaces the rule-as-authority slot per A2.05).
6. All applicable fields are committed atomically with the substrate content at the cell→substrate write per A2.10; the fields are queryable from substrate alone per A2.41.
7. Fields persist as long as the substrate content they describe persists per A2.08; automated processes do not strip, degrade, or normalize fields out of substrate state.

A substrate that fails any of (1)–(7) does not satisfy the commitment in the architectural sense.

9. Why naming the six-field commitment as standalone matters

Implementations under pressure to simplify substrate schemas, optimize for storage, or integrate with external audit infrastructure consistently drift toward partial provenance. The drift is steady because each missing field feels manageable in isolation. The cumulative effect is what breaks the commitment: accountability questions become partially answerable, paths fragment at missing antecedent references, conflicts accumulate as content rather than as first-class objects, and the substrate's source-of-truth status fails for the categories the missing fields support.

Naming the six-field commitment as standalone — with each field's content specified in section 2, the conditional structure clarified as architectural in section 3, and the failure modes enumerated in section 7 — gives downstream implementers a precise specification of what the substrate's provenance commitment requires. With this note and its companions A2.36–A2.39 drafted, the four accountability questions are fully formalized as standalone commitments, and the substrate-side metadata infrastructure that supports them is formalized as standalone alongside. Subsequent work that uses fewer fields, that flattens the conditional structure in either direction, or that locates fields outside substrate is not implementing the same commitment, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Six Provenance Metadata Fields per Substrate Write: The Architectural Infrastructure for Path Retracement in the Coordination Knowledge Substrate Pattern*. 4 May 2026. ORCID: 0009-0004-8065-3235.